COLEGIO UNIVERSITARIO IES

PARCIAL 2

Aplicaciones Web 1

Guadalupe Astudillo Pini

Materia: Aplicaciones Web 1

Docente: Andrés Senn

Carrera: Analista de sistemas

2026

FUNCIONALIDAD SELECCIONADA

Funcionalidad del TP4: Filtro del catálogo de productos

En este trabajo se analiza el funcionamiento del filtro del catálogo. Esta funcionalidad permite que el usuario modifique los criterios de búsqueda y vea en pantalla únicamente los productos que cumplen con las opciones seleccionadas.

Introducción

En este documento se analiza el funcionamiento del filtro del catálogo de productos del TP4. Se explica cómo comienza la interacción del usuario, cómo JavaScript procesa los datos y de qué manera se actualiza el contenido visible de la página mediante el DOM.

1. EJECUCIÓN DEL PROCESO

Explicación

La página `catalogo.html` carga el script `index.js` con `<script type="module" src="./recursos/js/index.js">`. Ese archivo identifica la página a partir de la clase del `<body>` (`pagina-catalogo`) y ejecuta la función `iniciarCatalogo()`, definida en `catalogo.js`.

El `<script>` se coloca al final del `<body>` para que primero se carguen los elementos HTML de la página y después se ejecute JavaScript. De esta forma, cuando el código intenta acceder a elementos del DOM, esos elementos ya se encuentran disponibles.

El atributo `type="module"` permite trabajar con distintos archivos JavaScript relacionados mediante importaciones y exportaciones.

Dentro de `iniciarCatalogo()` se referencian los elementos del DOM con `querySelector`, como el formulario, el contenedor `#catalogo` y el elemento donde se muestra el precio.

Luego se declara la función `actualizar` y se la asigna como manejador mediante `form.addEventListener("input", actualizar)`.

`addEventListener()` permite asociar una función a un evento. En este caso se utiliza el evento `input`, que se activa cuando el usuario modifica alguno de los controles del formulario. Por eso, cada cambio en los filtros vuelve a ejecutar la función `actualizar`.

El proceso se ejecuta en dos momentos. Primero, al cargar la página, cuando `iniciarCatalogo()` llama a `actualizar()` y muestra el catálogo completo. Después, cada vez que el usuario modifica un control del formulario, como la búsqueda, la categoría, el origen, el precio o el orden.

Captura de código

```
const actualizar = () => {
    const filtros = leerFiltros(form);
    if (salidaPrecio) salidaPrecio.textContent = "Hasta $" + filtros.precioMax.toLocaleString("es-AR");
    renderizar(contenedor, filtrarProductos(filtros));
};

form.addEventListener("input", actualizar);
```

Figura 1. Inicio de la ejecución de la funcionalidad.

2. PROCESAMIENTO DE DATOS

Explicación

Los productos se encuentran almacenados en un arreglo de objetos. Cada objeto representa un producto y contiene información como nombre, detalle, precio, categoría y origen.

Cuando el usuario utiliza los filtros, la función `filtrarProductos()` trabaja sobre ese arreglo y utiliza el método `filter()` para obtener solamente los productos que cumplen con las condiciones seleccionadas.

Se utiliza `filter()` porque permite crear un nuevo arreglo con los elementos que cumplen una condición, sin modificar el arreglo original.

En cambio, `map()` se utiliza cuando se quiere transformar cada elemento de un arreglo. En esta funcionalidad se usa durante el renderizado para convertir los productos en las tarjetas que luego se muestran en el catálogo.

Representación del dato en cada etapa del flujo:

1. Origen: `productos` es un arreglo de objetos. Cada objeto es un producto con propiedades como `id`, `nombre`, `detalle`, `precio`, `categoria` y `origen`.

2. Lectura de filtros: `leerFiltros()` toma los valores del formulario y devuelve un objeto. Sus propiedades son strings (el texto buscado y el orden), un número (`precioMax`, que se convierte con `Number()` porque el formulario entrega el valor como string) y las categorías y orígenes seleccionados.

3. Filtrado: `filtrarProductos()` encadena varios `filter()`. Cada uno recibe un arreglo y devuelve otro arreglo nuevo de objetos, que pasa al siguiente. El resultado final es un arreglo de objetos con los productos que deben mostrarse, que luego se ordena con `sort()` según la opción elegida. En el filtro de texto se usa `includes()` sobre un string.

4. Salida: `renderizar()` separa los productos por sección con `filter()` y, con `map()`, convierte cada objeto en una tarjeta, es decir, en un elemento del DOM.

Por qué no otros métodos: `forEach()` recorre el arreglo pero no devuelve nada, por lo que obligaría a crear un arreglo vacío y cargarlo con `push()` dentro del recorrido. `map()` devuelve un arreglo con la misma cantidad de elementos y solo transforma, por lo que no sirve para descartar productos. `find()` devuelve un único elemento, y el catálogo necesita una lista de resultados. `push()` agrega elementos a un arreglo existente, pero no selecciona cuáles conservar.

Captura de código

```
function filtrarProductos(filtros) {
    return productos
        .filter((p) => !filtros.texto || normalizar(p.nombre + " " + p.detalle).includes(filtros.texto))
        .filter((p) => filtros.categorias.size === 0 || filtros.categorias.has(p.categoria))
        .filter((p) => filtros.origenes.size === 0 || filtros.origenes.has(p.origen))
        .filter((p) => p.precio <= filtros.precioMax)
        .sort(ordenadores[filtros.orden]);
}
```

Figura 2. Procesamiento de los datos de la funcionalidad.

3. FUNCIONES

Explicación

En esta funcionalidad se utilizan funciones para organizar las distintas tareas del proceso.

`filtrarProductos()` es una función declarada que recibe los filtros seleccionados y devuelve los productos que cumplen con esas condiciones. Se utiliza como una función con una tarea específica dentro del proceso de filtrado.

También se utiliza `actualizar`, escrita como función flecha. Esta función agrupa las acciones que deben ejecutarse cada vez que se actualiza el catálogo: obtener los filtros, filtrar los productos y actualizar lo que se muestra en pantalla.

Se eligió función de declaración para `leerFiltros()`, `filtrarProductos()` y `renderizar()` porque son tareas con nombre propio: reciben parámetros, devuelven un resultado o realizan una acción concreta, y quedan definidas de forma independiente en el archivo.

`actualizar` se escribió como función flecha porque es el manejador del evento y se define dentro de `iniciarCatalogo()`. Al ser flecha, captura el scope donde fue declarada y puede usar `form`, `contenedor` y `salidaPrecio` sin recibirlos como parámetros. Además, las funciones flecha deben declararse antes de ser invocadas, por eso `actualizar` se define antes del `addEventListener`.

Los callbacks que reciben `filter()` y `map()` también son funciones flecha anónimas, ya que se escriben en el mismo lugar donde se pasan como argumento.

De esta manera, cada función cumple una tarea concreta dentro de la lógica del proceso.

Captura de código

```
function filtrarProductos(filtros) {
    return productos
        .filter((p) => !filtros.texto || normalizar(p.nombre + " " + p.detalle).includes(filtros.texto))
        .filter((p) => filtros.categorias.size === 0 || filtros.categorias.has(p.categoria))
        .filter((p) => filtros.origenes.size === 0 || filtros.origenes.has(p.origen))
        .filter((p) => p.precio <= filtros.precioMax)
        .sort(ordenadores[filtros.orden]);
}

> function renderizar(contenedor, lista) {…
}

export function iniciarCatalogo() {
    const form = document.querySelector(".buscador__form");
    const contenedor = document.querySelector("#catalogo");
    const salidaPrecio = document.querySelector("#precio-valor");

    const actualizar = () => {
        const filtros = leerFiltros(form);
        if (salidaPrecio) salidaPrecio.textContent = "Hasta $" + filtros.precioMax.toLocaleString("es-AR");
        renderizar(contenedor, filtrarProductos(filtros));
    };
```

Figura 3. Funciones utilizadas en la funcionalidad de filtrado.

4. SALIDA VISUAL – DOM

Explicación

JavaScript utiliza el DOM para acceder a los elementos de la página y actualizar el catálogo.

Para obtener una referencia a los elementos HTML se utiliza `querySelector()`. De esta manera, JavaScript puede trabajar con el formulario, el contenedor del catálogo y el elemento donde se muestra el precio.

La función `renderizar()` recibe el arreglo de productos filtrados y modifica el contenido del contenedor del catálogo para mostrar los productos correspondientes.

Cuando el usuario cambia un filtro, se vuelve a ejecutar el proceso y el contenido visible se actualiza sin necesidad de recargar la página.

Además, se utiliza `textContent` para modificar el texto que muestra el precio máximo seleccionado.

El archivo `catalogo.html` solo contiene el contenedor vacío `#catalogo`; las tarjetas se crean con JavaScript. El paso a paso es el siguiente:

1. `actualizar` modifica con `textContent` el texto del precio máximo ("Hasta $...").

2. `renderizar()` recibe el contenedor y el arreglo filtrado, y vacía el contenedor con `replaceChildren()` para eliminar las tarjetas dibujadas antes.

3. Si el arreglo está vacío, agrega un párrafo con el mensaje "No encontramos productos con esos filtros." y termina.

4. Si hay productos, recorre las secciones con `for...of`. Para cada una crea un `section` y un `div` de clase `grilla` mediante la función `crearElemento()`, que utiliza `createElement`.

5. Con `map(crearTarjeta)` genera una tarjeta por producto y las agrega a la grilla con `append()`. Luego agrega el título y la grilla al `section`.

6. Agrega cada `section` al contenedor `#catalogo`. El navegador muestra el resultado de inmediato, sin recargar la página (Figura 5).

Captura de código

```
function renderizar(contenedor, lista) {
    contenedor.replaceChildren();

    if (lista.length === 0) {
        contenedor.append(crearElemento("p", "catalogo__vacio",
            "No encontramos productos con esos filtros."));
        return;
    }

    for (const { id, titulo } of secciones) {
        const deLaSeccion = lista.filter((p) => p.seccion === id);
        if (deLaSeccion.length === 0) continue;

        const seccion = crearElemento("section", "seccion categoria");
        const grilla = crearElemento("div", "grilla");
        grilla.append(...deLaSeccion.map(crearTarjeta));
        seccion.append(crearElemento("h2", "seccion__titulo", titulo), grilla);
        contenedor.append(seccion);
    }
}

export function iniciarCatalogo() {
    const form = document.querySelector(".buscador__form");
    const contenedor = document.querySelector("#catalogo");
    const salidaPrecio = document.querySelector("#precio-valor");

    const actualizar = () => {
        const filtros = leerFiltros(form);
        if (salidaPrecio) salidaPrecio.textContent = "Hasta $" + filtros.precioMax.toLocaleString("es-AR");
        renderizar(contenedor, filtrarProductos(filtros));
    };

    form.addEventListener("input", actualizar);
    form.addEventListener("submit", (e) => e.preventDefault());
    form.addEventListener("reset", () => setTimeout(actualizar));

    actualizar();
}
```

Figura 4. Modificación del DOM.

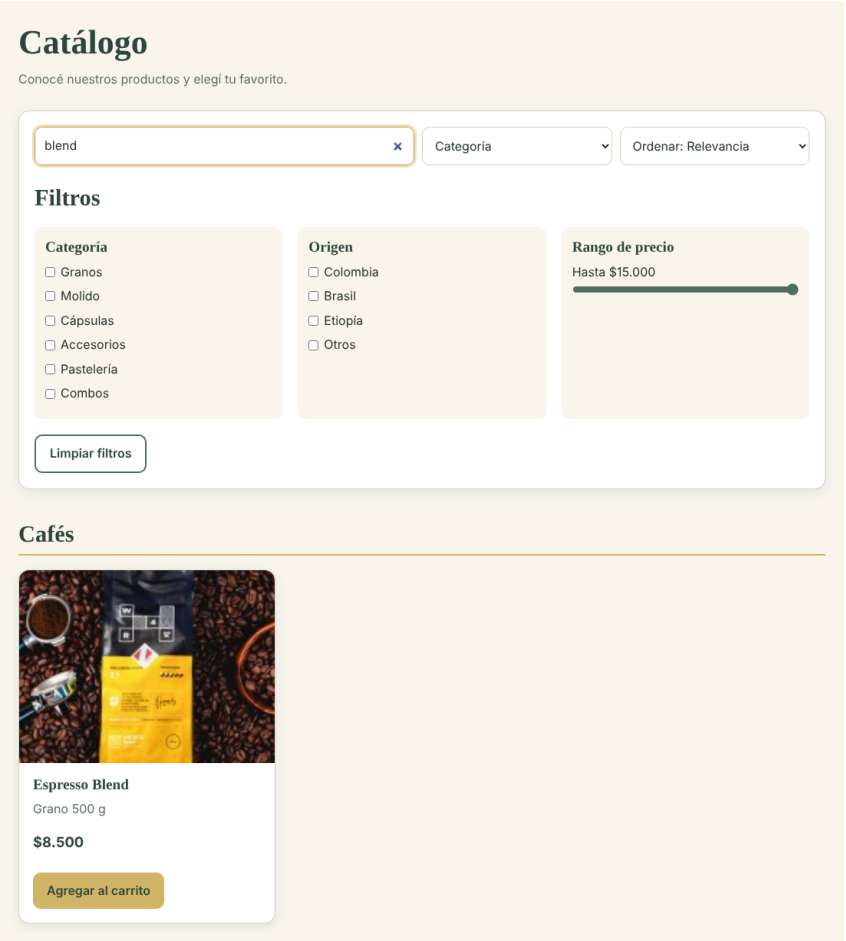


Figura 5. Resultado visible en el navegador: búsqueda "blend" en el catálogo (1 de 7 productos).

5. UBICACIÓN DEL CÓDIGO

Archivo JavaScript analizado: `src/recursos/js/vistas/catalogo.js`

Página HTML: `src/catalogo.html`

La página `catalogo.html` carga `index.js` y desde allí se ejecuta la función `iniciarCatalogo()` definida en `catalogo.js`.

Funcionalidad: Filtro del catálogo de productos.

CONCLUSIÓN

La funcionalidad seleccionada integra eventos, funciones, procesamiento de datos y modificación del DOM.

Al cargar la página se muestran inicialmente los productos del catálogo. Luego, cuando el usuario modifica alguno de los filtros, JavaScript procesa el arreglo de productos mediante `filter()` y actualiza el contenido visible.

De esta manera, el usuario puede visualizar solamente los productos que cumplen con los criterios seleccionados sin recargar la página.